CIÊNCIA DA COMPUTAÇÃO

Função main,

Diretivas e Bibliotecas

em Linguagem C

Profª. Drª. Tatiele B. Moro

AULA 9.1

```c
#include
<stdio.h>

int main()
{
  // ...
  return 0;
}
```

O que veremos hoje

AGENDA

01

Função main()

Estrutura, retorno e parâmetros

02

Diretivas #include

Incluindo arquivos e bibliotecas

03

Diretiva #define

Constantes e macros

04

Diretiva #undef

Removendo definições

05

Compilação Condicional

#ifdef, #if, #elif, #endif

Função main()

FUNDAMENTOS DA LINGUAGEM C

O que é a função main()?

A função main() é o ponto de entrada de todo programa em C. É obrigatória e única — sem ela, o programa não compila.

Todo programa C deve ter exatamente uma função main().

A execução começa na primeira instrução dentro de main().

Os comandos são delimitados por chaves { }.

```c
// Estrutura básica
int main()
{

  // lógica do programa
  return 0;
}
```

⚠ Ponto de entrada

Sem main(), o linker não sabe onde iniciar a execução do programa.

Retorno da Função main()

COMUNICAÇÃO COM O SISTEMA OPERACIONAL

`int main()`	`return 0;`	`return N;`
Declara que main() retornará um valor inteiro para o SO	Indica que o programa terminou com SUCESSO	Onde N ≠ 0 indica um CÓDIGO DE ERRO ao SO

```c
#include <stdio.h>
int main()
{
    printf("Olá, Mundo!\n");
    return 0; // sucesso
}
```

Parâmetros da Função main()

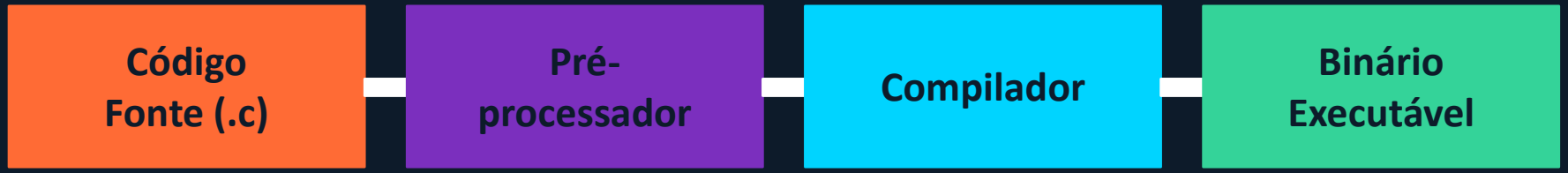
PROTÓTIPOS POR SISTEMA OPERACIONAL

C ANSI (Padrão)	`int main(void)`
C ANSI (Padrão)	`int main(int argc, char **argv)`
UNIX-like / Windows	`int main(int argc, char **argv, char **envp)`
Apple (MacOS/iOS)	`int main(int argc, char **argv, char **envp, char **apple)`

💡 *O protótipo da main() varia conforme o sistema operacional — o padrão C ANSI define as versões mais portáveis.*

Diretivas do Pré-processador

O QUE ACONTECE ANTES DA COMPILAÇÃO



Iniciam com #
Toda diretiva começa com o símbolo # (hash/cerquilha).

Não geram código
As diretivas existem no código-fonte, mas não no código compilado.

Execução anterior
São processadas ANTES do compilador ver o código.

Diretiva #include

INCLUINDO ARQUIVOS NO CÓDIGO-FONTE

#include <arquivo.h>

1. Usa < > para arquivos do sistema

O pré-processador procura na pasta include do sistema (ex: /usr/include no Linux).

2. Usa " " para arquivos locais

O pré-processador procura na pasta atual do projeto. Se não encontrar, busca na pasta include.

3. A diretiva é substituída pelo conteúdo

O arquivo incluído literalmente substitui a linha #include no código.

```
// Biblioteca padrão (< >)
#include <stdio.h>
#include <string.h>

// Arquivo local (" ")
#include "minhaBib.h"
#include "utils.h"
```

Extensão .h

Arquivos .h são cabeçalhos (headers) que contêm declarações de funções, tipos e constantes reconhecidas pelo compilador.

Diretiva #define

DEFININDO CONSTANTES SEM GASTAR MEMÓRIA

```
#define  NOME_CONSTANTE
         valor
```

```c
#include <stdio.h>
#define PI 3.141592653589793
int main() {
    double raio = 2.0;
    double area = PI * raio * raio;
    printf("Área = %f\n", area);
    return 0;
}
```

Sem uso de memória

O pré-processador substitui textualmente o identificador antes de compilar. Nenhuma variável é alocada.

Sem sinal de =

Não use o operador de atribuição (=). O #define é uma substituição de texto, não uma declaração de variável.

Por convenção: MAIÚSCULAS

Constantes definidas com #define são escritas em letras maiúsculas para diferenciá-las de variáveis.

#define como Macro de Código

SUBSTITUIÇÃO DE TRECHOS COMPLETOS

Além de constantes simples, #define pode substituir trechos inteiros de código.

Definição

```c
#include <stdio.h>
#define ERRO\
    printf("Erro!\n");
int main()
{
    ERRO;
    return 0;
}
```

Após pré-processamento

```c
int main()
{
    // ERRO foi substituído
    printf("Erro!\n");
    return 0;
}
```

💡 *O identificador ERRO é substituído pelo printf antes da compilação.*

#define com Parâmetros (Macros)

FUNÇÕES INLINE SEM OVERHEAD

```c
#include <stdio.h>
#define SOMA(x,y) ((x) + (y))
int main() {
    int a = SOMA(1, 2);
    double b = SOMA(1.0, 2.0);
    printf("a = %d\n", a);
    printf("b = %f\n", b);
    return 0;
}
```

⚠ Sem espaços!

Correto:

```c
#define SOMA(x,y)
```

Errado:

```c
#define SOMA (x,y)
```

Vantagem

Funciona para qualquer tipo: int, double, float. Não há overhead de chamada de função.

Cuidado!

Macros fazem substituição textual. Parênteses incorretos podem gerar resultados inesperados!

Perigo: Parênteses em Macros

POR QUE O RESULTADO PODE SURPREENDER

Problema sem parênteses

```
#define SOMA(x,y)  x + y
printf("%d\n", 10 * SOMA(1,2));
// Expandido: 10 * 1 + 2  →  12 ❌
// Esperado: 10 * (1 + 2)  →  30
```

Solução: sempre envolva parâmetros e expressão com parênteses

```
#define SOMA(x,y)  ((x) + (y))
printf("%d\n", 10 * SOMA(1,2));
// Expandido: 10 * ((1) + (2))  →  30 ✅
```

Macro PRODUTO: Outro Exemplo

PROTEGENDO EXPRESSÕES COM PARÊNTESES

❌ **Sem proteção**

```
#define PRODUTO(x,y)  (x * y)
printf("%d\n", PRODUTO(2+3, 4));

// Expande para:
// (2+3 * 4) = 14 ❌
```

✅ **Com proteção total**

```
#define PRODUTO(x,y)  ((x) * (y))
printf("%d\n", PRODUTO(2+3, 4));

// Expande para:
// ((2+3) * (4)) = 20 ✅
```

Regra de ouro para macros com parâmetros:
`#define MACRO(x,y)  ((x) op (y))` — sempre envolva cada parâmetro E a expressão completa com parênteses.

Diretiva #undef

REMOVENDO DEFINIÇÕES

#undef NOME_IDENTIFICADOR

Finalidade: remover uma definição criada com #define para redefinir ou limitar seu escopo.

```
#define TAM_STRING 20
// ... código usando TAM_STRING = 20 ...
#undef TAM_STRING
// Redefine com novo valor
#define TAM_STRING 100
```

Redefinição controlada

Use #undef antes de redefinir uma constante para evitar warnings do compilador.

Escopo de definições

Limite o uso de uma definição a apenas um trecho do código, removendo-a depois.

Compilação Condicional

INCLUINDO CÓDIGO APENAS QUANDO NECESSÁRIO

Permite que partes do código sejam compiladas somente se determinadas condições forem satisfeitas. Essencial para depuração e portabilidade.

#ifdef `#ifdef DEBUG`

Se a macro FOI definida

#ifndef `#ifndef WINDOWS`

Se a macro NÃO FOI definida

#if `#if DEBUG == 1`

Se a expressão for verdadeira

#elif `#elif DEBUG == 2`

Senão, se outra condição

#else `#else`

Caso contrário (nenhuma anterior)

#endif `#endif`

Fecha o bloco condicional

Exemplo: #ifdef e #else

MODO DEBUG vs PRODUÇÃO

```c
#include <stdio.h>
#define DEBUG
int main()
{
    #ifdef DEBUG
        printf("Descricao detalhada: ...\n");
    #else
        printf("Nenhuma descricao\n");
    #endif
}
```

Com #define DEBUG

O bloco #ifdef é incluído na compilação.
→ Imprime: "Descricao detalhada: ..."

Sem #define DEBUG

O bloco #else é incluído.

→ Imprime: "Nenhuma descricao"

Removendo ou comentando #define DEBUG, você muda o comportamento sem alterar a lógica.

Exemplo: #if, #elif e #endif

MÚLTIPLOS NÍVEIS DE DEPURAÇÃO

```c
#include <stdio.h>
#define DEBUG 0
int main()
{
    #if DEBUG == 1
        printf("Descricao detalhada: ...\n");
    #elif DEBUG == 2
        printf("Descricao resumida: ...\n");
    #else
        printf("Nenhuma descricao\n");
    #endif
}
```

DEBUG = 1

→ "Descricao detalhada: ..."

DEBUG = 2

→ "Descricao resumida: ..."

DEBUG = 0

→ "Nenhuma descricao"

Basta mudar o valor de DEBUG para selecionar o nível de verbosidade desejado.

Resumo das Diretivas

VISÃO GERAL DE TUDO QUE ESTUDAMOS

Diretiva	Uso principal	Exemplo
`#include <...>`	Inclui biblioteca do sistema	`#include <stdio.h>`
`#include "..."`	Inclui arquivo local	`#include "utils.h"`
`#define`	Define constantes e macros	`#define PI 3.14`
`#undef`	Remove uma definição	`#undef PI`
`#ifdef / #ifndef`	Compila se (não) definido	`#ifdef DEBUG`
`#if / #elif / #else`	Compila condicionalmente	`#if DEBUG == 1`
`#endif`	Fecha bloco condicional	`#endif`

Boas Práticas

DICAS PARA ESCREVER CÓDIGO C PROFISSIONAL

01

Sempre inclua apenas o necessário

Incluir bibliotecas desnecessárias aumenta o tempo de compilação.

02

Use #define para constantes

Evite 'números mágicos' no código. Use constantes nomeadas com #define.

03

Parênteses em macros sempre!

Envolva parâmetros e expressão: #define MACRO(x) ((x) * 2)

04

return 0 indica sucesso

Por convenção, retorno 0 = sucesso. Qualquer outro valor = erro.

05

Use compilação condicional p/ debug

Isole código de depuração em blocos #ifdef DEBUG para fácil remoção.

06

Nomes MAIÚSCULOS para #define

Convenção universal: constantes e macros em caixa alta.

O QUE APRENDEMOS

Conclusão

- main() é o coração de todo programa C — ponto de entrada obrigatório.
- #include conecta seu código a bibliotecas poderosas como stdio.h.
- #define cria constantes e macros sem consumir memória em runtime.
- Parênteses são essenciais em macros para garantir comportamento correto.
- Compilação condicional controla o que é compilado com #ifdef e #if.